MySQL InnoDB로 이해하는 트랜잭션

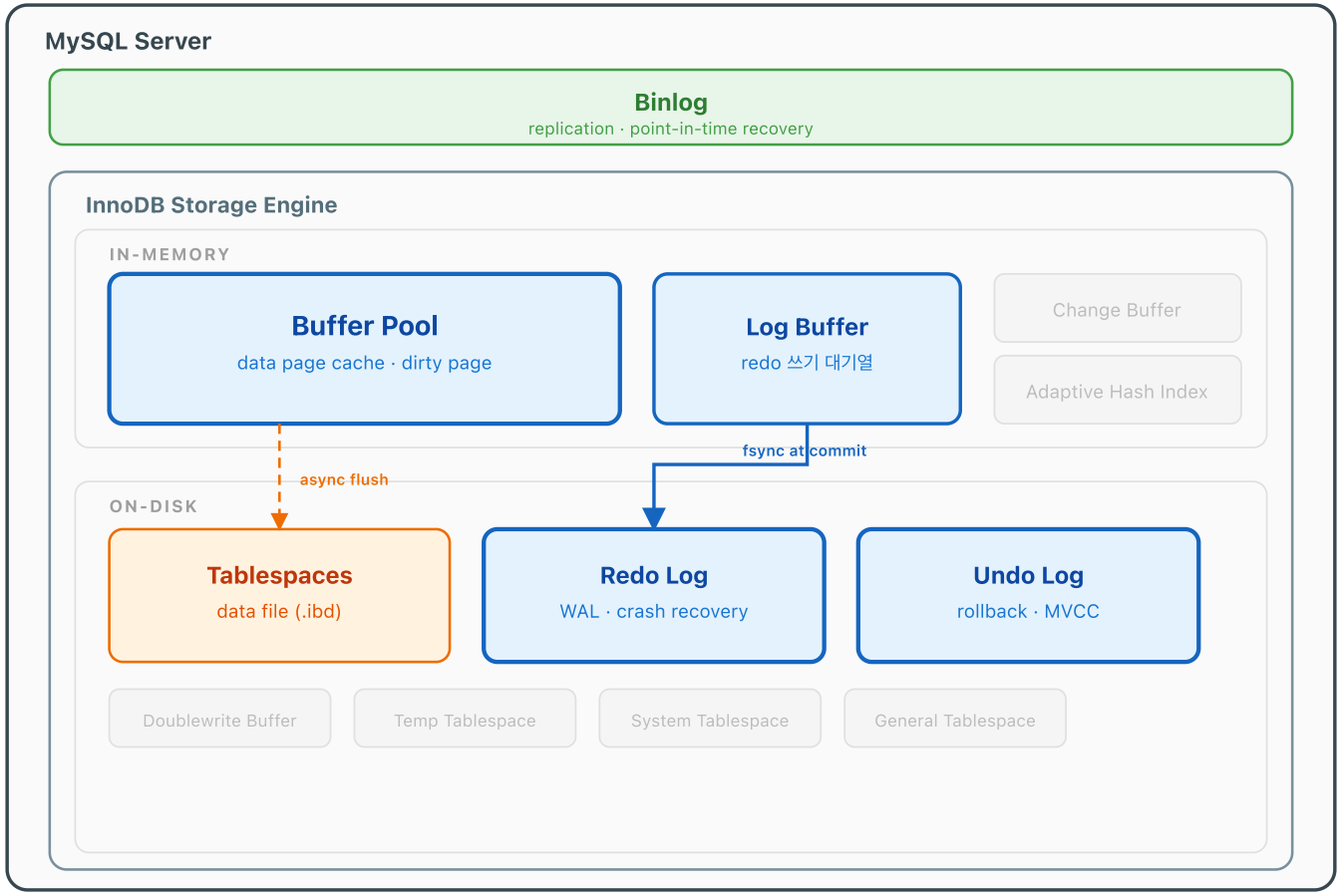
InnoDB · Lock · Isolation · MVCC

이 질문에 답할 수 있는가?

- `UPDATE` 쿼리가 성공했을때 **disk에는 뭐가 써져 있는가?**
- `UPDATE` 쿼리는 얼마나 **lock을 잡는가?**
- row가 겹치지 않는 두 트랜잭션이 **deadlock 걸릴 수 있는가?**
- 트랜잭션이 길어지면 **왜 성능이 떨어지는가?**
- 서버가 죽으면 데이터는 **어떻게 복구되는가?**

InnoDB

InnoDB Architecture



Redo Log

commit 때 **data file 전체를 먼저 쓰는 건 아니다.**

- data file에 바로 반영하려면 여러 page를 disk에 써야 한다
- 그 사이 crash가 나면 일부 page만 반영될 수 있다

그래서 변경 사실을 **redo log에 기록하고 commit 시 fsync로 durability를 확보**한다.

Buffer Pool

table/index page를 메모리에 캐시하는 영역

- **읽기**: disk I/O 없이 메모리에서 처리
- **쓰기**: 메모리에서 수정 후 나중에 반영 (dirty page)

redo log 덕분에 dirty page를 즉시 flush하지 않아도 crash recovery가 가능하다.

InnoDB가 처리하는 대부분의 읽기·쓰기는 이 메모리 영역에서 일어난다.

Checkpoint

redo log는 계속 쌓이지만 **무한히 들고 있을 필요는 없다.**

checkpoint는 **이 시점 이전의 변경은 이미 data file에 반영되었음**을 의미한다.

그래서 recovery는 checkpoint 이후 redo만 보면 된다.

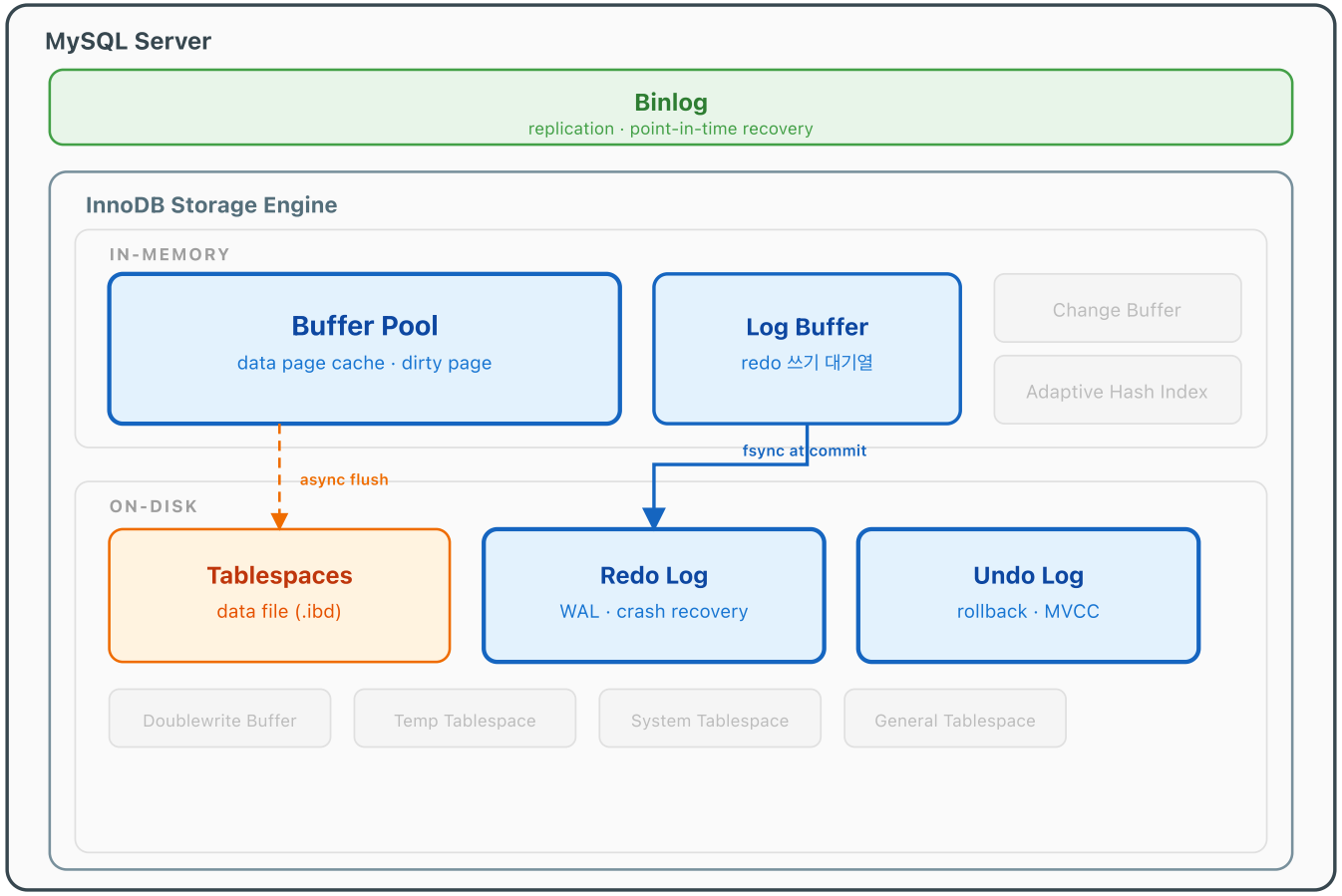
Undo Log

commit 전에 실패하면 **이미 Buffer Pool에서 수정된 page**를 되돌려야 한다.

변경 전 값을 별도로 보존해둔다. 이것이 **Undo Log**.

undo log는 rollback뿐 아니라 **MVCC**에서 과거 버전을 만들 때도 사용된다.

다시 보면


MySQL Server
Binlog
replication · point-in-time recovery
InnoDB Storage Engine
IN-MEMORY
Buffer Pool
data page cache · dirty page
Log Buffer
redo 쓰기 대기열
Change Buffer
Adaptive Hash Index
fsync at commit
async flush
ON-DISK
Tablespaces
data file (.ibd)
Redo Log
WAL · crash recovery
Undo Log
rollback · MVCC
Doublewrite Buffer
Temp Tablespace
System Tablespace
General Tablespace

예시) UPDATE 쿼리

겉으로는 `UPDATE` 한 줄이지만,

안에서는 **같은 row를 찾고 → 잠그고 → 수정하고 → commit** 한다.

이 과정에서

Buffer Pool, Undo Log, Redo Log, Data File이 차례로 관여한다.

상태 1. 계획

상태 1. 파싱 / 최적화 / 인덱스 결정

MySQL Server가 SQL을 해석하고 실행 계획을 만든다.

1. 계획 | 2. 탐색/잠금 | 3. 수정 | 4. COMMIT | 5. 그 후

Client: 대기

MySQL Server

SQL 파싱 / 최적화

실행 계획 생성 사용 인덱스 결정

Lock

아직 변화 없음

Buffer Pool

아직 변화 없음

Undo Log

아직 변화 없음

Redo Log

아직 변화 없음

Data File

아직 변화 없음

핵심: 어떤 인덱스를 쓸지 결정하는 단계다. 아직 InnoDB page 수정은 시작하지 않는다.

상태 2. 탐색 / 잠금

상태 2. page 탐색 / disk read / lock

선택한 인덱스를 따라 필요한 page를 찾고, 대상 record에 lock을 건다.

1. 계획 | **2. 탐색/잠금** | 3. 수정 | 4. COMMIT | 5. 그 후

Client: 대기

MySQL Server

선택한 실행 계획 전달

Lock

index record lock

스캔한 대상 record 잠금

Buffer Pool

필요한 page 조회

없으면 Data File에서 read

Undo Log

아직 기록 안 함

Redo Log

아직 기록 안 함

Data File

read source

Buffer Pool miss 때만 읽음

핵심: 수정 전 준비 단계다. page를 찾고, 필요하면 disk에서 읽고, lock을 잡는다.

상태 3. 수정

상태 3. undo 기록 / page 수정 / redo 기록

변경은 먼저 메모리와 로그에 반영된다. data file은 아직 그대로일 수 있다.

1. 계획 | 2. 탐색/잠금 | **3. 수정** | 4. COMMIT | 5. 그 후

Client: 대기

MySQL Server

대기

Lock

lock 유지

수정 완료 전까지 유지

Buffer Pool

page 수정

data page가 dirty page가 됨

Undo Log

변경 전 정보 기록

rollback / MVCC 용

Redo Log

log buffer 기록

변경 내용 기록

Data File

아직 미반영

핵심: 변경 전 값은 undo, 변경 내용은 redo, 실제 page 수정은 Buffer Pool에서 일어난다.

상태 4. COMMIT

상태 4. COMMIT / OK 응답

COMMIT 때 redo durability를 확보하고, 그다음 클라이언트가 OK를 받는다.

1. 계획 | 2. 탐색/잠금 | 3. 수정 | **4. COMMIT** | 5. 그 후

Client: OK 응답

MySQL Server

COMMIT 처리

redo durability 확인 후 OK

Lock

해제 직전 / 해제

Buffer Pool

dirty page 유지

Undo Log

기록 보관 중

Redo Log

redo log file fsync

이제 crash recovery 가능

Data File

아직 미반영 가능

핵심: COMMIT 성공의 기준은 redo log의 durable write다. data file은 아직 안 바뀌었을 수 있다.

상태 5. 그 후

상태 5. flush / checkpoint / purge

응답 후에는 data file 반영과 undo 정리가 나중에 이어진다.

1. 계획 | 2. 탐색/잠금 | 3. 수정 | 4. COMMIT | **5. 그 후**

Client: 이미 OK 받음

MySQL Server

요청 처리 완료

Lock

이미 해제됨

Buffer Pool

dirty page flush

Data File로 반영

Undo Log

purge 대상 정리

오래된 undo 제거

Redo Log

checkpoint 전진

이전 redo 공간 재사용

Data File

flush 후 반영 완료

핵심: data page flush는 나중에 일어나고, purge가 오래된 undo를 정리한다.

MVCC

Multi-Version Concurrency Control

UPDATE 다음에 바로 나오는 질문

방금 본 대로 `UPDATE`는

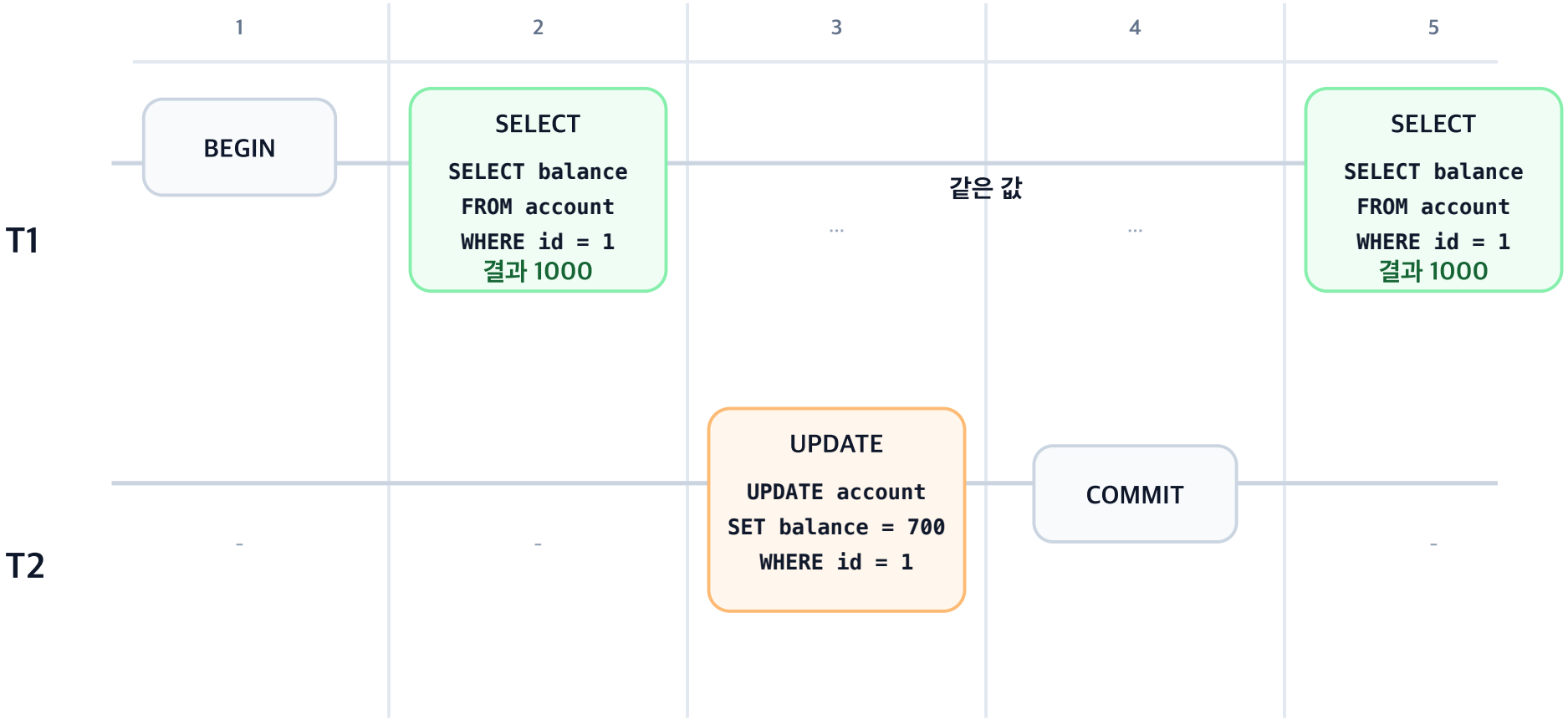
- Buffer Pool의 page를 먼저 바꾸고
- undo에 **변경 전 값**을 남기고
- redo로 commit durability를 보장한다

그러면 일반 `SELECT`는 뭘 읽어야 할까?

- **지금 Buffer Pool의 최신 값?**
- 아니면 **내가 읽어야 하는 과거 시점의 값?**

이 문제를 푸는 게 **MVCC**다.

Consistent Nonlocking Reads



MVCC가 하는 일

> **일반 `SELECT` 를 lock 없이 처리하면서도**
> **트랜잭션이 읽어야 하는 시점의 값은 지켜준다.**

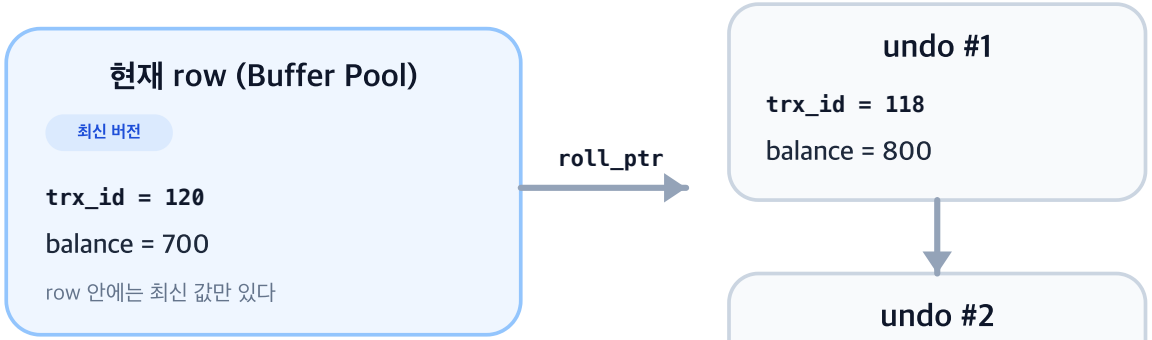
- writer는 최신 값을 계속 써도 되고
- reader는 자기 시점의 버전을 읽으면 된다

즉, **읽기와 쓰기가 서로 덜 막히게 만든다.**

Undo Chain

Undo Chain

현재 row가 너무 최신이면, undo를 따라가며 이전 버전을 복원한다.



읽는 방법

1. 현재 row가 보이는지 확인
2. 안 보이면 undo #1 확인
3. 여전히 안 보이면 더 과거로 이동

리턴되는 값

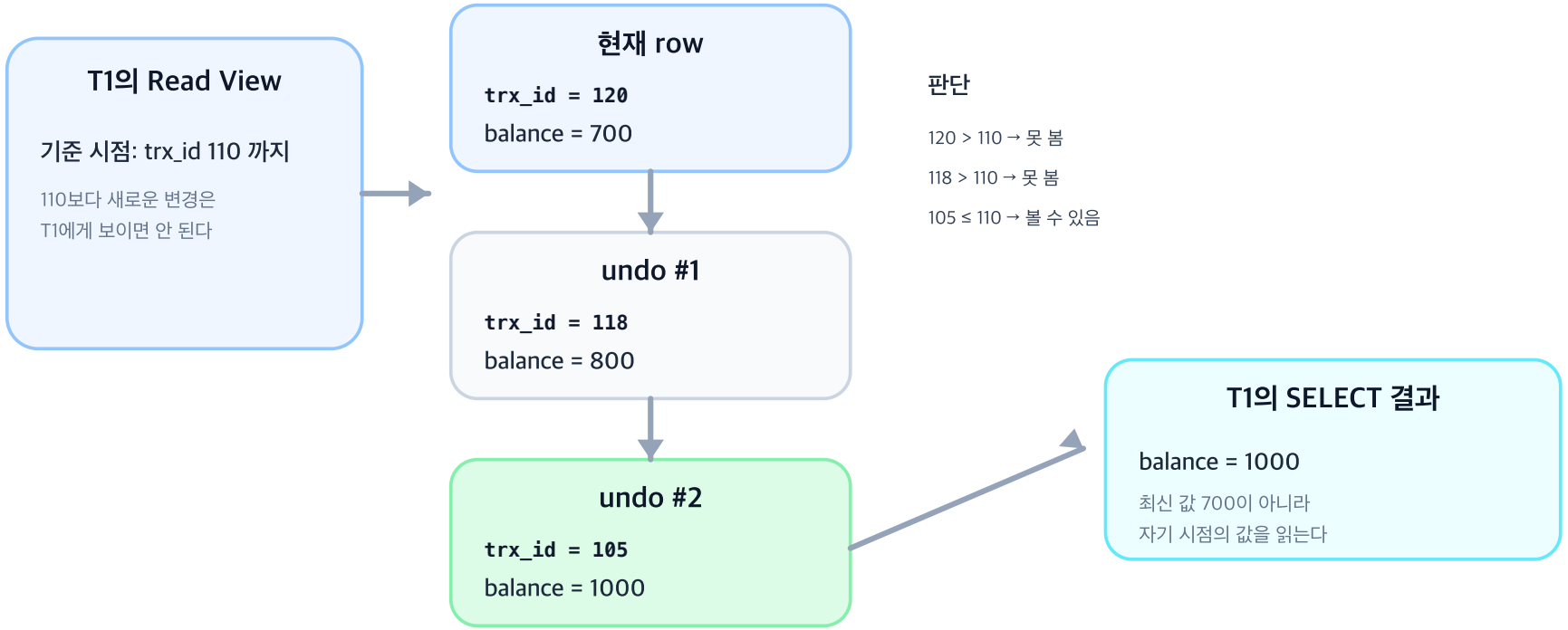
내가 볼 수 있는 가장 최신 버전

핵심: 현재 row를 그대로 읽는 것이 아니라, 필요하면 undo로 이전 버전을 만들어 읽는다.

Read View + Undo Chain

Read View + Undo Chain

REPEATABLE READ에서는 첫 read 때의 시점을 기준으로, 보이는 버전이 나올 때까지 undo를 따라간다.



핵심: Buffer Pool에 최신 값이 있어도, read view + undo chain 덕분에 REPEATABLE READ가 성립한다.

격리수준

격리수준

- MySQL 기본값은 **REPEATABLE READ**
- **REPEATABLE READ**의 일반 `SELECT`는 **MVCC**로 처리된다
- 그래서 트랜잭션 안에서 **같은 읽기 결과를 유지**할 수 있다

격리수준은 동시에 실행되는 트랜잭션 사이에서
어떤 데이터 버전을 읽게 할지와 **어떻게 충돌을 제어할지** 정하는 규칙이다.

MVCC와 Lock의 역할은 다르다

- **일반** `SELECT`

  → MVCC

  → 같은 시점의 값을 읽는다

  → phantom이 **안 보이게** 한다

- `SELECT ... FOR UPDATE` / `UPDATE` / `DELETE`

  → Lock

  → 충돌을 직접 제어한다

  → phantom이 **생기지 않게** 막는다

즉, **읽기 일관성은 MVCC가**, **충돌 제어는 lock이** 맡는다.

Lock

Shared Lock과 Exclusive Lock

	Shared Lock (S)	Exclusive Lock (X)
Shared Lock (S)	호환	충돌
Exclusive Lock (X)	충돌	충돌

- 일반 `SELECT` → **MVCC** (lock 없음)
- `SELECT ... FOR SHARE` → **S lock**
- `SELECT ... FOR UPDATE`, `UPDATE`, `DELETE` → **X lock**

이 두 요청이 동시에 들어온다. 어떻게 될까?

	1. 먼저 확인한다	2. 없으면 INSERT 한다
요청 A	`SELECT * FROM user_point WHERE user_id = 42 FOR UPDATE` 결과: 0건	`INSERT INTO user_point (user_id, point) VALUES (42, 100)`
요청 B	`SELECT * FROM user_point WHERE user_id = 45 FOR UPDATE` 결과: 0건	`INSERT INTO user_point (user_id, point) VALUES (45, 100)`

Lock은 index record에 걸린다

InnoDB는 **row 자체가 아니라 index record**를 잠근다.

- **인덱스가 있으면**

  → 필요한 record만 lock

- **인덱스가 없으면**

  → 조건 확인을 위해 전부 스캔

  → 스캔한 record 전부 lock

즉, **인덱스는 조회 성능뿐 아니라 lock 범위도 결정한다.**

Gap Lock

아까 예시에서 `user_id = 42`는 인덱스에 없다.

`FOR UPDATE`로 0건을 확인한 뒤 다른 트랜잭션이 INSERT하면

같은 조건에서 결과가 달라질 수 있다.

InnoDB는 이를 막기 위해

row가 들어올 수 있는 index 구간을 잠근다.

```
idx_user_id:  ... | 30 | 40 |         | 50 | 80 | ...
                              ↑
                          gap (40, 50)
```

InnoDB(RR)의 **locking read / 쓰기**는 이 gap에 **gap lock**을 건다.

→ 이 구간에 INSERT되는 것을 막는다.

Gap Lock은 무엇을 막나

gap lock의 목적은 **다른 gap lock을 막는 것**이 아니라,
그 구간에 새 row가 INSERT되는 것을 막는 것이다.

- 같은 gap에 여러 트랜잭션이 **gap lock**을 잡을 수 있다
- 대신 그 gap 안으로 들어오려는 **INSERT**는 막힌다

같은 gap에서	결과
gap lock + gap lock	함께 가능
gap lock + INSERT	INSERT 대기

다시 보면

idx_user_id

`... | 30 | 40 | gap (40, 50) | 50 | 80 | ...`

	1. SELECT ... FOR UPDATE	2. INSERT 시도
요청 A	**`user_id = 42 FOR UPDATE`** 0건 확인 → gap lock (40, 50)	**`INSERT user_id = 42`** insert intention lock 필요 → B의 gap lock과 충돌
요청 B	**`user_id = 45 FOR UPDATE`** 0건 확인 → gap lock (40, 50)	**`INSERT user_id = 45`** insert intention lock 필요 → A의 gap lock과 충돌

DEADLOCK 발생

정리하면

- 출발점은 **동시성**이다. 여러 트랜잭션이 같은 데이터를 동시에 만진다.
- 그래서 DB는 **어떤 데이터 버전을 읽게 할지**와 **어떻게 충돌을 제어할지**를 정해야 한다.
- 그 규칙이 **격리수준**이고, MySQL 기본값은 **REPEATABLE READ**다.
- **redo log는 durability**를, **undo log는 rollback과 MVCC**를, **buffer pool은 성능**을 담당한다.

다음단계

- 같은 트랜잭션 안에서 `SELECT` 한 값과 `UPDATE` 가 읽는 값이 **항상 같은가?**
- 아까 deadlock 예시는 **어떻게 해결하는가?**
- 격리수준 종류는 무엇이 있고, **각각 무엇을 포기하고 무엇을 얻는가?**
- binlog와 redo log는 둘 다 변경 기록이다. **정합성은 어떻게 맞추는가?**
- Aurora는 InnoDB를 쓰지만, storage 구조에서 **무엇이 다른가?**
- `innodb_flush_log_at_trx_commit` 을 바꾸면 **정확히 뭘 포기하는가?**